Part 7 — Building the strain: assembly, everyday assays and genome engineering (weeks 20 to 25)

I said when I handed over part 6 that a written module adds least to the technique weeks, because a protocol you can run beats a chapter about it. That is true of the wet-lab steps and false of everything around them, which is what this part covers: the arithmetic that tells you whether an assembly will work before you order it, the controls that make a daily assay interpretable, and the probability calculations that decide how many MAGE cycles or how much DNA a project needs. None of that is in a protocol, and all of it is the difference between running a technique and understanding one.

So these chapters deliberately do not teach you to pipette. They teach the five things a protocol leaves out: why a method exists, what its failure mode is, what number tells you in advance whether it will work, which control makes its output mean something, and when to choose a different method. Where there is code — the sequence work in chapter 21 — it is written to be typed in, because a design check should be a script that runs in a second rather than an afternoon with a sequence viewer.

One structural note. Weeks 20 to 25 are where the module's earlier parts stop being theory. The control analysis of part 2 chapter 15 says you must change several steps at once, and MAGE in chapter 23 is how; the leak argument of part 1 chapter 8 says a repressor sets a floor, and CRISPRi in chapter 24 is how far down that floor actually goes; the restriction barriers of part 4 chapter 52 decide whether chapter 25's non-model host is reachable at all. The cross-references are not decoration; they are the same problems arriving with tools attached.

Ch.	Week	Chapter
20	20	DNA assembly: four methods, and how to choose between them
21	21	Multigene pathways in code: cutting geometry, overhang design and in silico verification
22	22	Everyday assays: what the daily measurements actually mean
23	23	Genome engineering I: recombineering, MAGE, and Cas9 as a counterselection
24	24	Genome engineering II: CRISPRi, base editing, biosensors and dynamic control
25	25	Genome design and non-model hosts: edit counts, error budgets and toolkit arithmetic

DNA assembly: four methods, and how to choose between them

The methods differ in one thing that matters for pathway work: what they leave behind at the junction, and whether you can take the construct apart again afterwards.

Scars, and why they are the organising idea

A scar is sequence left at a junction by the assembly chemistry itself — a restriction site, a linker, anything that is there because of how the parts were joined rather than because the design wanted it. In a single-gene construct a scar is cosmetic. In a multi-part construct it is three problems at once.

- **It occupies a position where sequence matters.** Between a ribosome binding site and a start codon, or inside the 5′ untranslated region, the local sequence sets translation initiation — part 1 chapter 8 — so a scar there is not neutral, it is an uncontrolled part.
- **It repeats.** The same scar at every junction means repeated sequence in one molecule, which is a substrate for recombination and therefore for the instability of part 1 chapter 10.
- **It consumes a site.** If the scar is a restriction site, that enzyme can no longer be used elsewhere in the construct, which compounds as the construct grows.

The four methods line up on exactly that axis, and the comparison below is the week's deliverable.

Method	How it works	Junction	Best for	Fails when
Restriction–ligation	Cut both pieces with the same enzyme, ligate the compatible ends	Leaves the recognition site — a scar by construction	A single subclone into an existing vector; moving a cassette between standard backbones	The insert contains the site; more than two or three parts; sequence at the junction matters
Gibson assembly	One isothermal reaction: a 5′ exonuclease chews back to expose single strands, the overlaps anneal, a polymerase fills, a ligase seals	Scarless — the junction is whatever you designed	Joining a few large fragments, any sequence, PCR-generated ends; circularising a construct	Overlaps are short, repetitive or structured; many fragments at once; parts must be swapped repeatedly
Golden Gate	A Type IIS enzyme cuts *outside* its recognition site, so the site is removed with the cut and the four-base overhang is user-chosen; cutting and ligation run in one pot because correct joins destroy the site	Scarless, with a defined four-base junction	Many parts in a defined order, one reaction, high efficiency	Parts contain internal sites; overhangs are badly chosen (chapter 21)
MoClo and similar hierarchies	Golden Gate with a standard: fixed overhang syntax for part types, two enzymes alternating by level, parts to transcription units to multigene constructs	Defined and standard	Combinatorial libraries; any construct whose parts will be swapped repeatedly; foundry work	The project is one-off, or the parts cannot be domesticated cheaply

Gibson, read as chemistry

Three enzymes in one tube at 50 °C, each doing a job you should be able to state. **T5 exonuclease** chews the 5′ ends back, exposing 3′ single-stranded overhangs on both fragments — it is also heat-labile, which is why the reaction self-terminates and why the order of events works out. **The annealing** itself is not enzymatic: the exposed complementary overlaps find each other. **A high-fidelity polymerase** fills the gaps left behind, and **Taq ligase** seals the nicks, chosen because it is thermostable and discriminates against mismatched nicks.

Practical envelope: overlaps of 15 to 40 bp, with 20 to 25 the usual choice, and reliability falling off at both ends. Too short and the annealing is unstable at reaction temperature; too long and you are paying for primer length and increasing the chance that an overlap is similar to another one in the mixture. The failure modes are specific: GC-rich or structured overlaps that form hairpins instead of annealing, repeated sequences that let fragments join in the wrong order, and more than four or five fragments at once, where the number of possible wrong pairings outgrows the right one.

Golden Gate, and the trick that makes one pot work

A Type IIS enzyme such as BsaI recognises an asymmetric site and cuts at a fixed distance *outside* it. Two consequences follow and together they are the whole method. First, **the overhang sequence is yours** rather than the enzyme's, so you can choose a different four-base overhang at every junction and thereby fix the order of parts. Second, **a correct**

ligation destroys the recognition site, because the site sat outside the cut and ends up in the discarded fragment; so a correctly assembled junction cannot be cut again while an incorrectly assembled one — or an unreacted part — can. Running cutting and ligation together in one tube with cycling temperatures therefore pushes the reaction toward the correct product, which is why efficiencies are high enough to join six or ten parts in one reaction.

A four-overhang scheme joining five parts in a defined order, as the week asks you to draw: part A carries overhang 1 on its left (matching the vector) and 2 on its right; part B carries 2 and 3; C carries 3 and 4; D carries 4 and 5; E carries 5 and 6, with 6 matching the vector's other end. Each junction is unique, so A cannot ligate to C and no part can insert backwards. Chapter 21 is about choosing those six four-base sequences so that they behave, which turns out to be less obvious than it looks.

MoClo, and why foundries standardised

MoClo adds a syntax to Golden Gate: each part type — promoter, 5′ untranslated region, coding sequence, terminator — has fixed, agreed overhangs, so any promoter from the collection drops into any construct in the promoter position. Levels alternate enzymes, typically BsaI to assemble parts into transcription units and BsmBI to assemble units into multigene constructs, so a level-2 reaction cannot cut its own products.

The reason biofoundries standardised on this is not elegance, it is logistics. A standard syntax makes parts interchangeable so that a library of promoters can be combined with a library of coding sequences without redesigning anything; every reaction runs at the same temperature with the same enzyme mix, so a robot can do them all; and the design step becomes a computation over part identifiers rather than a bespoke primer design. The cost is paid up front, in domesticating every part to remove internal sites, which chapter 21 quantifies.

Choosing, with the week's two questions answered

Six parts, each to be swapped independently later. MoClo-style hierarchical Golden Gate, and the deciding word is *independently*. Scarless matters because some junctions are in translation-initiation territory; defined overhangs matter because six parts in a defined order in one reaction is exactly what the method is for; and the standard syntax matters most of all, because "swapped later" means you are building a family of constructs, so the up-front domestication cost is amortised over every future variant. Gibson would assemble the first construct about as well and would require new primers for every swap.

The one case for restriction–ligation. When you are moving a single cassette between two backbones that already carry compatible sites, no junction sequence matters, and the construct will not be rebuilt. It needs no primers, no sequence design and no domestication; it is twenty minutes of work with reagents already in the freezer. The mistake is not using it, it is using it for a multi-part pathway and then discovering that the scars are in the untranslated regions and that two of your parts contain the site.

One habit that applies whichever method you choose: design the verification before you order anything. For each junction, name the primer pair that would amplify across it and the band size you expect, and know which junctions a gel can confirm and which need sequencing — chapter 22 draws that line. A construct that assembles and cannot be verified cheaply is a construct you will not trust.

Scar	`sequence left by the chemistry, not the design; repeats, occupies regulatory space, consumes a site`
Gibson	`T5 exonuclease + polymerase + Taq ligase, 15–40 bp overlap, scarless`
Type IIS	`cuts outside its recognition site → user-chosen 4 bp overhang, and the site is destroyed by a correct join`
One-pot logic	`correct products cannot be re-cut, so cycling drives the reaction toward them`
Choose by	`how many parts, whether junction sequence matters, and whether parts will be swapped again`

Three things people get wrong

- ‘Scarless is a nicety.’ A scar in a 5′ untranslated region is an uncontrolled part, and the same scar at ten junctions is ten repeats inviting recombination.
- ‘Golden Gate works because Type IIS enzymes are efficient.’ It works because a correct junction destroys the site, so the reaction is driven rather than merely fast.
- ‘Gibson handles any number of fragments.’ Reliability falls off past four or five, because wrong pairings grow faster than right ones.

Self-check — answer on paper first

1. Explain in two sentences why a Type IIS enzyme permits one-pot assembly while a Type II enzyme does not.

A Type IIS enzyme cuts outside its recognition sequence, so the site is carried away on the discarded fragment and a correctly ligated junction no longer contains it — the product is therefore immune to further cutting while unreacted parts are not, which lets cutting and ligation proceed together and drives the equilibrium toward the assembly. A Type II enzyme cuts within its site, so every correct ligation recreates a substrate for the enzyme and the reaction cannot be run in one pot without the product being destroyed as fast as it forms.

2. You must join three 4 kb fragments amplified from genomic DNA, once, with no plan to rebuild. Which method and why?

Gibson. The parts are PCR products, so overlaps can be added in the primers at no extra step; three fragments is well within its reliable range; it needs no domestication, which matters because genomic fragments of 4 kb will almost certainly contain internal Type IIS sites — an expected 1.5 to 2 per 4 kb for a 6-base recogniser, counting both strands; and since nothing will be rebuilt, the standardisation that would justify MoClo buys nothing.

3. Why is a repeated junction sequence a stability problem rather than only an aesthetic one?

Because identical sequences in one molecule are substrates for homologous recombination, which can delete everything between two copies in the same orientation or invert it if they are opposed. In a multigene construct with the same scar at every junction, the most likely recombination product is a shorter construct missing one or more genes, and because that product has lost the expression burden it frequently outgrows the intact one — part 1 chapter 10's instability argument, arriving through the assembly design rather than through the host.

What to open a book for. Addgene's "Plasmids 101: Golden Gate cloning", which is short and better than any textbook treatment, and Gibson et al., Nat. Methods 6:343 (2009), read for the enzymology rather than the protocol. A MoClo or Yeast Toolkit methods paper for the syntax. No book is needed this week.

Multigene pathways in code: cutting geometry, overhang design and in silico verification

A design check should be a script that runs in a second, not an afternoon in a sequence viewer. This week is where sequences stop being pictures and become data you can test.

Sequences as data

Biopython gives you three objects and that is most of what pathway design needs: `Seq` for a sequence with biological methods on it, `SeqRecord` for a sequence with features and annotations, and `SeqIO` to read and write FASTA and GenBank. The useful habit is to put features into a pandas DataFrame — one row per feature, columns for type, start, end and strand — at which point every check you want to run is a query rather than a visual scan.

```
from Bio import SeqIO
import pandas as pd

rec = SeqIO.read("plasmid.gb", "genbank")
rows = [{"type": f.type, "start": int(f.location.start), "end": int(f.location.end),
         "strand": f.location.strand,
         "name": f.qualifiers.get("label", [""])[0]} for f in rec.features]
df = pd.DataFrame(rows).sort_values("start")

for f in rec.features:                      # check every CDS translates to its annotation
    if f.type == "CDS":
        prot = f.extract(rec.seq).translate(to_stop=True)
        print(f.qualifiers.get("label", ["?"])[0], len(prot))
```

That last loop is worth running on any plasmid you are handed. An annotation that disagrees with its own sequence is common, usually because someone edited the sequence in a viewer without updating the feature, and finding it before you build is free.

Cutting geometry, drawn once and then coded

Draw BsaI on paper until it is automatic: it recognises GGTCTC, cuts one base after the site on the top strand and five after it on the bottom, leaving a four-base 5′ overhang whose sequence is whatever you put in those four positions. BsmBI recognises CGTCTC with the same one-and-five spacing; BbsI recognises GAAGAC with two-and-six. The spacing is the only thing that differs, and it is why the enzymes are interchangeable in a design as long as the spacer bases are right.

Then the thing people forget: **a recognition site exists on both strands**. The reverse complement of GGTCTC is GAGACC, and a site in that orientation cuts just as well, on the other side. Any site-finding function that searches only the given strand will miss half of them, and the symptom is an assembly that fails for no visible reason.

```
from Bio.Seq import Seq

def find_sites(seq, site="GGTCTC", spacer=1, overhang=4):
    """Return (position, strand, overhang) for every occurrence on both strands."""
    s, hits = str(seq).upper(), []
    rc = str(Seq(site).reverse_complement())
    for i in range(len(s) - len(site) + 1):
        if s[i:i+len(site)] == site:
            a = i + len(site) + spacer
            hits.append((i, "+", s[a:a+overhang]))
        if s[i:i+len(site)] == rc:
            b = i - spacer - overhang
            hits.append((i, "-", str(Seq(s[b:b+overhang]).reverse_complement())))
    return hits
```

Cross-check it against `Bio.Restriction`, and expect a discrepancy that is not an error: Biopython reports *cutting* positions while the function above reports *recognition* positions. Reconciling the two by hand once is the exercise, because confusing them is how a domestication edit gets made in the wrong place.

Domestication, and how often you will need it

Removing an internal site means changing a base without changing the protein — a synonymous codon substitution — then confirming by translation that the protein is identical. How often does this come up? For a six-base recogniser, the

expected number of sites on both strands of a sequence of length L is $2L/4^6$:

Part length	Expected BsaI sites (both strands)	Probability of none
1 kb	0.49	61%
3 kb	1.46	23%
30 kb pathway	14.6	effectively zero

So a typical 3 kb coding sequence has a three-in-four chance of needing domestication for one enzyme, and a ten-gene pathway will contain around fifteen sites between the two enzymes a MoClo scheme uses. This is the up-front cost that chapter 20 said standardisation charges, and it is why parts collections exist: the domestication is done once, by someone, and then never again.

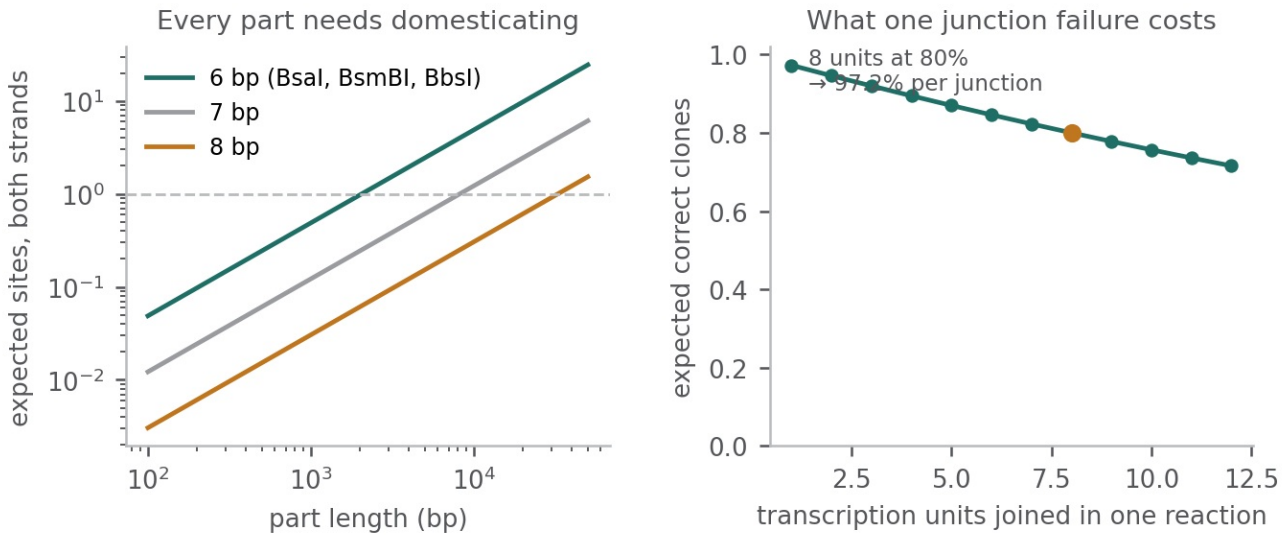


Figure 21.1 Left: expected recognition sites on both strands against part length for three motif lengths — a 3 kb part has about a three-in-four chance of containing a six-base site. Right: expected fraction of correct clones against the number of transcription units joined in one reaction, at the per-junction fidelity implied by 80% success for eight units. Computed for this module; the left panel assumes random equal base composition, so count your actual sequence rather than trusting it.

Overhang design, which is where assemblies silently fail

Four-base overhangs are not interchangeable. T4 DNA ligase joins some pairs faithfully and others promiscuously, and the comprehensive profiling of all four-base overhang ligations established the two practical facts: fidelity varies substantially between overhangs, and **mismatched pairs do ligate**, particularly when they differ at only one position. So a set of overhangs chosen because they look different can give an assembly that produces mostly wrong products, with no diagnostic other than disappointing colonies.

The rules a checker should enforce, and the arithmetic behind how many overhangs you can have:

- No palindromes. A palindromic overhang is its own reverse complement, so a part can join to itself or insert backwards. There are 16 palindromic four-mers out of 256.
- No duplicates and no reverse-complement pairs in the set, which leaves 120 mutually non-complementary overhangs as the absolute maximum.
- At least two base differences from every other overhang *and* from every other overhang's reverse complement, because one-base mismatches are the ones that ligate. Imposing that by greedy search collapses the maximum from 120 to **24** — which is why published high-fidelity sets contain roughly that many, and why a set of thirty overhangs chosen by eye cannot be faithful.
- A complexity filter: no homopolymers, no extreme GC content, since both correlate with poor ligation.

Write the checker, then hand-pick seven overhangs and run it; the usual outcome is that two of your seven are one base apart from each other's complement, which is exactly the failure the fidelity data describe. Generate the set by search instead, and assign the result to the junctions of your transcription unit as a table — part, left overhang, right overhang — which becomes the specification you order against.

Yeast in vivo assembly, and what a connector does

Yeast assembles DNA by homologous recombination: give it fragments whose neighbours share 40 to 60 bp of overlap

and it will join them in order inside the cell, including the vector, in one transformation. The method's strength is that it does not care about internal restriction sites and can join many fragments; its weakness is that each junction is a recombination event with its own probability, and that similar overlaps mispair.

The hybrid approach — build transcription units in vitro by Golden Gate, then join the units in yeast through standard connector sequences — exists because each method fails where the other works. Golden Gate struggles when the construct grows to many units because the number of unique faithful overhangs is limited, as the arithmetic above shows; yeast recombination struggles when the fragments are many and similar, which transcription units sharing promoters and terminators inevitably are. A connector is a short, designed, non-repetitive sequence placed at the ends of each unit, whose only job is to be the unique homology that pairs that unit with its neighbour — which removes the mispairing between similar units and makes the junction count the only remaining limit. The 2026 paper from KRIBB that the tracker names reports up to eight transcription units assembled at 80% efficiency in one transformation this way.

Now the arithmetic the week asks for, which is the useful part. If eight units at 80% overall implies independent junctions, the per-junction success is $0.8^{1/8}$ = **97.2%**. Under that figure, three units give 92%, five give 87%, eight give 80% and twelve would give 72%. Two readings follow. The method is remarkably good — 97% per junction is excellent — and the decline with size is gentle enough that the limit is not efficiency but screening: at 80% correct you pick four colonies and expect three to be right.

What the independence model leaves out, and you should say so whenever you quote it: mispairing between overlaps that are similar rather than identical, which is not independent across junctions because one mispairing event consumes two junctions at once; recombination between repeated sequences *within* the assembled construct, such as the same terminator used in four units, which produces clones that assembled correctly and then deleted part of themselves; and point mutations in correct-looking clones, which no junction PCR will detect. The model gives you the floor, not the answer.

Checking the build before you order it

The week's last session is the habit worth keeping for the rest of your career: assemble the construct in code, from the part sequences, in the order and with the overlaps you designed, and then test the assembled sequence. Translate every open reading frame and confirm each junction reads in frame. Scan the final molecule for stray BsaI and BsmBI sites, since domestication is only complete once the assembled sequence — including the junctions you created — is clean. List the colony-PCR primers for each junction with expected band sizes. Then order.

This costs an hour and catches the errors that cost a month: a part cloned in the wrong orientation because its overhangs were palindromic, a frameshift at a junction where the overhang length was miscounted, a site reintroduced by the junction itself, and a verification strategy that turns out to give indistinguishable band sizes for the right and wrong products.

Site frequency	`expected sites ≈ 2L·(1/4)^k; 1.46 for a 6-mer in 3 kb`
BsaI geometry	`GGTCTC, cut +1 top / +5 bottom → 4 bp 5′ overhang; reverse strand is GAGACC`
Overhang set limit	`256 four-mers → 16 palindromes → 120 non-RC → 24 with ≥2 differences from all others and their complements`
Junction success	`overall = p^n; 80% at 8 units → p = 0.972; 5 units → 87%`
Before ordering	`assemble in silico, translate every ORF, rescan for sites, list verification primers`

Three things people get wrong

- ‘I searched the sequence for the site.’ On one strand. The reverse complement cuts too.
- ‘The overhangs are all different.’ Different is not enough; one-base mismatches ligate, so the requirement is at least two differences from every other overhang and its complement.
- ‘The assembly worked, so the construct is right.’ Junction PCR confirms topology, not sequence. Point mutations and internal recombination survive it.

Self-check — answer on paper first

1. A colleague designs a 12-part Golden Gate assembly with 13 overhangs chosen by eye. What do you predict and how would you check?

That some fraction of the products will be mis-assembled, because thirteen overhangs chosen without a distance constraint will very likely contain at least one pair differing by a single base from each other or from a reverse complement, and such pairs ligate. The check is the set checker: no palindromes, no duplicates or reverse-complement pairs, and a minimum Hamming distance of two against every other overhang and every other overhang's complement.

If it fails, replace the offending overhangs from a published high-fidelity set — there are about 24 mutually compatible ones, which is enough for twelve parts but not by a wide margin.

2. Derive the per-junction success rate implied by 60% correct clones for five transcription units, and say what it predicts for eight.

With five junctions treated as independent, $p = 0.6^{1/5} = 0.902$, so eight units would give $0.902^8 = 43\%$. The practical reading is that at 43% you still only need to screen a handful of colonies, so efficiency is not the barrier; what the lower per-junction figure suggests is a systematic problem — overlaps too short, similar overlaps mispairing, or repeated internal sequences — which is worth fixing because it will get worse with every unit added.

3. Why should the final assembled sequence be rescanned for Type IIS sites even if every part was domesticated?

Because a junction creates new sequence that did not exist in either part: the four-base overhang plus the flanking bases of both neighbours can spell a recognition site that neither part contained. The same applies to any sequence feature you care about — a cryptic promoter, a terminator, an internal ribosome binding site or a start codon out of frame — which is why the in silico assembly is scanned as a whole rather than part by part.

What to open a book for. The Biopython Tutorial's `Bio.Seq`, `Bio.SeqIO` and restriction chapters, read with an editor open. Potapov et al., ACS Synth. Biol. (2018) for the ligation-fidelity result that justifies the overhang rules; Lee et al., ACS Synth. Biol. (2015) for the Yeast Toolkit syntax; and Seong et al., Trends Biotechnol. (2026) for the connector approach and the eight-unit figure.

Everyday assays: what the daily measurements actually mean

These are the measurements a lab makes every day, and not recognising them marks someone who has not been at a bench. The thread through all of them: every assay has a control that makes it interpretable, and the first question about any result is which controls were run.

qPCR, and what a fold change rests on

The measurement is a cycle threshold, C_t: the cycle at which fluorescence crosses a threshold, which is later for a less abundant template. Everything else is arithmetic resting on one assumption — how efficiently each target amplifies.

Efficiency comes from a standard curve of C_t against the log of template concentration. Its slope gives $E = 10^{-1/\text{slope}} - 1$, so a slope of −3.32 is 100% efficiency (a perfect doubling per cycle), −3.60 is 90%, and −3.10 is 110% — the last usually meaning inhibition at high template rather than better-than-perfect chemistry.

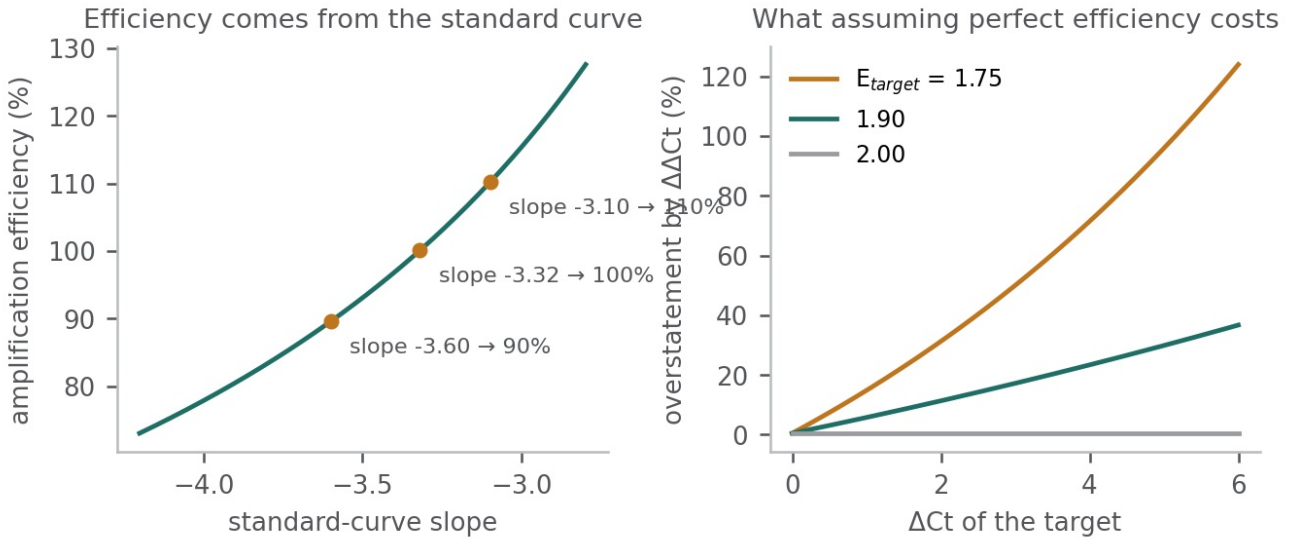


Figure 22.1 Left: amplification efficiency from the standard-curve slope, with three slopes marked. Right: how much the $\Delta\Delta C_t$ method overstates a fold change when the target does not amplify at 100% efficiency, against the size of the change, computed against the Pfaffl correction with a reference efficiency of 2.05. Computed for this module.

The $\Delta\Delta C_t$ method assumes both target and reference amplify at exactly 100%, so a fold change is $2^{-\Delta\Delta Ct}$. The Pfaffl method uses the measured efficiencies instead. The difference is not academic: for a ΔC_t of 3 with a target efficiency of 1.90 against a reference of 2.05, $\Delta\Delta C_t$ reports 6.96-fold where Pfaffl reports 5.94 — **17% overstated**; at a target efficiency of 1.75 it reports 6.96 against 4.64, **50% overstated**. And because the error compounds with the exponent, it grows with the size of the change you are reporting, which is the opposite of convenient. The rule: measure both efficiencies once per assay pair and use $\Delta\Delta C_t$ only when they agree with each other and with 100%.

The controls, and what each rules out. **No-template control**: contamination and primer dimers. **No-reverse-transcriptase control**: genomic DNA contamination, which otherwise appears as expression and is the commonest reason an "expressed" gene is not. **Melt curve**: specificity — one product gives one peak, and a second peak or a low-temperature shoulder means dimers or a second amplicon. **Standard curve**: efficiency and linear range. And the reference gene must be **validated in your own conditions**, because a gene that is stable across growth phases in one organism on one medium need not be stable across induction, nutrient limitation or your own pathway's burden — which is exactly what you are perturbing. Normalising to a reference that itself responds to induction converts a real change into no change, or manufactures one.

Gels, read as evidence

An agarose gel is a cheap, fast answer to one question: is there DNA of about the right size? Read the common outcomes:

- **A band at the wrong size** — mispriming, a wrong template, or a construct that is not what you think. If it is

consistently wrong by the same amount, suspect the construct rather than the PCR.

- **A smear** — degraded template, too much template, or over-cycling; also what sheared genomic DNA looks like.
- **Two bands where one was expected** — mispriming, a heteroduplex, a partially digested plasmid, or a genuine mixture, which in colony PCR often means a mixed colony.
- **A bright front** — primer dimers, which also consume reagents and depress the real product; the no-template control tells you immediately.

Failed PCR against failed gel is distinguished by the ladder and a positive control: if the ladder resolves, the gel ran; if a known-good template amplifies in the same run, the reaction chemistry works. And the line this draws for chapter 21's verification: a gel confirms the **presence and approximate size** of a junction, so it can tell you a part is in and roughly where. It cannot tell you the sequence, so a junction whose correctness depends on four bases of overhang, a reading frame, or a point mutation needs sequencing. The sensible division is colony PCR to triage, sequencing to confirm the clone you will keep.

Protein: present, soluble, active — three different claims

Running soluble and insoluble fractions side by side on SDS-PAGE answers the question a total lysate cannot: part 1 chapter 2's first hypothesis, that the protein is there and in inclusion bodies. A strong band in the insoluble lane and nothing in the soluble one is the classic plant-enzyme-in-*E. coli* result, and it points at expression temperature, induction strength, chaperone co-expression or a different host — the triage of week 30.

A Western blot establishes that something of approximately that size binds your antibody. It does not establish quantity without a loading control and a standard curve, and it does not establish activity at all. Three reasons a band can be present while the enzyme is dead, all from part 1 chapter 2: it is misfolded, aggregated or proteolysed into a fragment the antibody still binds; it is missing a cofactor, a metal or a redox partner; or it requires a post-translational modification or disulfide the host cannot make. The assay to add is an activity assay — initial rate, in the linear range, with a blank — because activity is the only measurement that answers the question you actually have.

Cells: distributions, not averages

Flow cytometry plots one event per cell, so a dot plot of two parameters and a histogram of one show you the *distribution*. Gating — drawing the boundary that defines a population — is subjective, and two competent people gate the same data differently, which is why gates must be shown and justified rather than described. Autofluorescence sets the floor, so an unlabelled control is not optional.

What a population average hides is the shape: a culture whose mean fluorescence doubled may have doubled uniformly, or may have split into an induced and an uninduced subpopulation — a bimodal response, which is common with positive feedback and with leaky inducible systems, and which means something entirely different for a biosensor-based screen. This is the connection to the biosensor work of week 26: a sensor is useful for sorting only if its single-cell distribution separates producers from non-producers, and a mean shift with overlapping distributions cannot sort.

Microscopy gives an image, and an image is not a measurement until it is quantified with stated rules. Brightfield and phase contrast show morphology and visible inclusion bodies — which is a real and useful observation, since inclusion bodies are large enough to see as refractile poles; fluorescence shows localisation. What magnification buys is resolution of structures, not certainty, and the honest limit on a fluorescence image of cells with inclusion bodies is that you can conclude the protein is aggregated and localised, not how much of it there is or whether the soluble remainder is active.

Finally the three growth measurements, which part 1 chapter 10 introduced and which disagree legitimately. Optical density is light scattering, so it tracks biomass and cell size rather than number. Colony counts measure cells able to grow on that medium, missing viable-but-non-culturable cells. Live/dead staining measures membrane integrity, which is neither growth nor number. For a yield calculation, use dry weight or a calibration to it; for the course of a culture, optical density; for viability after a stress, colony counts, with staining as the fast proxy you validated against them.

Reading a results figure like a reviewer

The week's last session is a habit: for each figure, write what was measured, which controls appear, and what claim it actually supports. Then the question you would ask in a lab meeting. A short list of the questions that are nearly always worth asking: what is the error bar and is it SD or SE (part 2 chapter 17); how many biological replicates, and were they independent colonies; what is the loading control or reference gene and was it validated here; where is the no-template or unlabelled control; is this an endpoint or an initial rate; and is the axis a log scale that flatters a small difference.

And the one you should ask of your own planned work: which artefact would most easily be mistaken for my result? For a strain-engineering project the usual answers are a growth-rate difference being read as a productivity difference, a measurement drift aligned with sample order, and a construct that recombined — and each has a control you can run in

advance rather than a defence you mount afterwards.

qPCR efficiency	`E = 10^(−1/slope) − 1; slope −3.32 → 100%`
$\Delta\Delta C_t$ error	`17% overstated at E = 1.90, 50% at E = 1.75, for a` ΔC_t `of 3`
Required controls	`no template, no RT, melt curve, standard curve, validated reference gene`
Gel can confirm	`presence and size; sequence needs sequencing`
Band ≠ activity	`soluble/insoluble fractions, then an initial-rate assay`
Average ≠ distribution	`a bimodal population and a uniform shift give the same mean`

Three things people get wrong

- '$\Delta\Delta C_t$ is the standard method.' It is, and it assumes both assays amplify perfectly; at 1.75 efficiency it overstates a threefold ΔC_t change by half.
- 'The reference gene is a housekeeping gene.' It is a hypothesis about stability that must be validated in the conditions you are perturbing.
- 'The mean fluorescence went up.' Which is consistent with every cell responding a little and with a third of them responding a lot — and only one of those can be sorted.

Self-check — answer on paper first

1. Your target amplifies at 85% efficiency and your reference at 100%. What does $\Delta\Delta C_t$ do to a reported sixfold induction, and what do you do instead?

It overstates it, because the true per-cycle multiplication of the target is 1.85 rather than 2, so each cycle of ΔC_t represents less change than assumed, and the error compounds with the exponent — for a ΔC_t near 2.6 the overstatement is of order 30 to 40%. Use the Pfaffl formulation, which raises each measured efficiency to its own ΔC_t, and report the efficiencies alongside the fold change. Better still, redesign the target assay: an efficiency of 85% usually means primer or amplicon problems that are worth fixing rather than correcting for.

2. A Western blot shows a strong band for your enzyme and the lysate has no activity. Give three hypotheses and the experiment for each.

Misfolding into inclusion bodies — run soluble and insoluble fractions side by side, since the blot of a total lysate cannot distinguish them. A missing cofactor, metal or redox partner — add the candidate cofactor to the assay or co-express the partner. Degradation to a fragment that retains the epitope, or the wrong protein cross-reacting — check the band size precisely against a marker and confirm identity by mass spectrometry or by a tagged construct. A fourth, often the real one, is that the assay conditions are wrong for this enzyme, which is cheapest to test by varying pH and temperature before concluding anything.

3. Why can optical density, colony counts and live/dead staining disagree without any of them being wrong?

Because they measure three different things. Optical density measures light scattering, which depends on biomass and cell size, so filamentous or enlarged cells read high while dividing into more, smaller cells can read lower. Colony counts measure the ability to form a colony on that medium, so stressed but living cells that cannot grow under those conditions are missed. Live/dead staining measures membrane integrity, which a cell can retain while being metabolically dead or lose transiently. The question decides which to trust: mass balances need dry weight, viability after stress needs counts, and the course of a culture needs optical density.

What to open a book for. No book: the MIQE guidelines for reporting qPCR, the Pfaffl paper for the efficiency-corrected formulation, and the literatures on reference-gene validation and on Western blot loading controls. Read them alongside four figures from your own paper log, one of each type, which is the exercise that makes them stick.

Genome engineering I: recombineering, MAGE, and Cas9 as a counterselection

Three things get confused in every discussion of CRISPR in bacteria: cutting, editing and regulating. Cas9 cuts. Recombineering edits. In a bacterium the cut is not the edit — it is the selection that finds it.

Recombineering: short homology, because a phage does the work

Lambda Red is three phage proteins that let a linear DNA fragment with short homology arms replace a chromosomal sequence. Each has one job and you should be able to state all three. **Exo** is a 5′→3′ exonuclease that chews back one strand of the incoming double-stranded fragment, exposing 3′ single-stranded ends. **Beta** is a single-strand annealing protein that binds those ends and pairs them with complementary sequence at the replication fork. **Gam** inhibits the host's RecBCD nuclease, which would otherwise degrade the linear fragment before anything else could happen.

That mechanism is why 40 to 50 bp of homology suffices where classical homologous recombination needs a kilobase: the pairing is done by a dedicated protein at a single-stranded region of a replication fork, not by the host's homology search. Practically, it means the homology arms fit on the PCR primers, so a knockout cassette is one PCR from a template.

The one-step inactivation workflow, drawn end to end: design primers with 40 to 50 bp of homology to the regions flanking your target gene, followed by sequence annealing to a resistance cassette flanked by FRT sites; amplify the cassette; transform the PCR product into cells expressing Red from an inducible plasmid; select on the antibiotic; verify by colony PCR with primers outside the homology arms, which gives a different band size for the wild type and the replacement; then, if you want the marker out, express FLP recombinase to excise it, leaving a scar, and cure both plasmids by growth at the non-permissive temperature. Each step has a failure mode worth knowing: no colonies usually means the Red system was not induced or the PCR product was contaminated with template plasmid, which is why the template carries a different marker and why DpnI digestion of the PCR is standard.

MAGE: many loci, by cycling oligonucleotides

MAGE drops the double-stranded cassette and the selection. Synthetic single-stranded oligos, each about 90 bases with the desired change in the middle, are introduced by electroporation into cells expressing Beta alone; Beta anneals them at the replication fork as Okazaki-fragment mimics, so the change is incorporated into one daughter strand. No selection is applied. Then the cycle repeats — grow, electroporate, grow — and because many different oligos can be introduced in the same electroporation, many loci are edited in parallel across a population.

What limits it is arithmetic, and the arithmetic is the point of this chapter. If the per-cycle allele-replacement frequency at a locus is p, then after N cycles the probability that a given locus is edited in a given cell is $1 - (1 - p)^N$, and for k independent loci the probability that all of them are edited is that quantity to the power k.

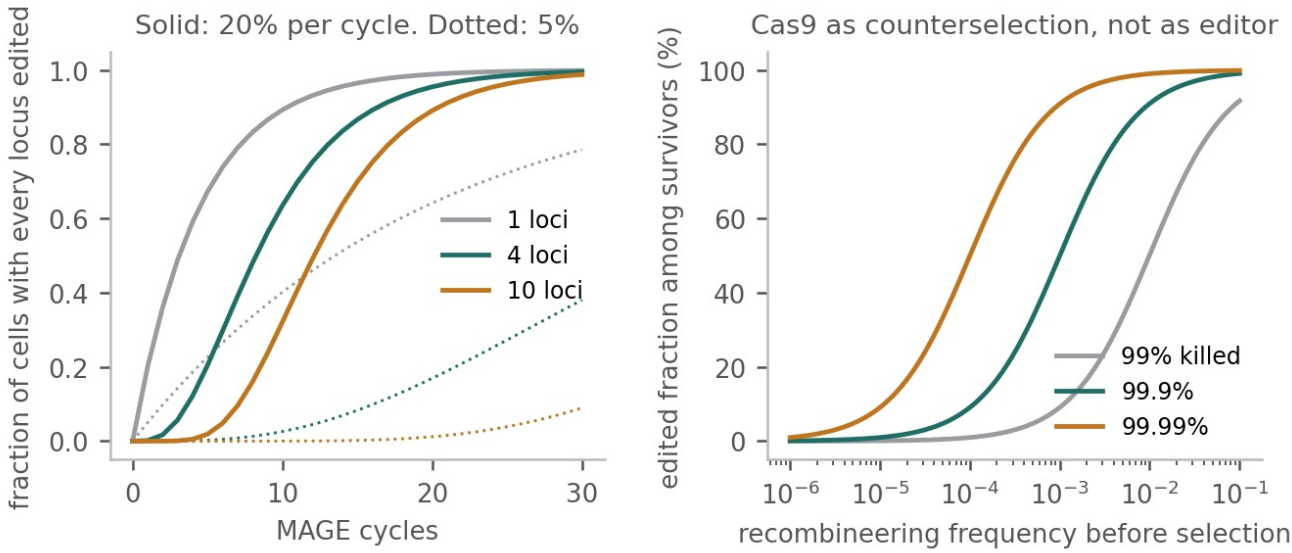


Figure 23.1 Left: fraction of the population with every target locus edited against MAGE cycles, at 20% per-cycle replacement (solid) and 5% (dotted), for one, four and ten loci. Right: the fraction of survivors that carry the intended edit when a Cas9 cut is used to kill unedited cells, against the

recombineering frequency, for three killing efficiencies. Computed for this module; the per-cycle frequency and the killing efficiency are the assumptions, and both vary with oligo design and strain.

At a realistic 20% per cycle: four loci need ten cycles to reach 64% of the population and five cycles give only 20%; ten loci give 32% at ten cycles and need twenty to reach 89%. At 5% per cycle — a poorly designed oligo, a secondary-structured target, or a strain with active mismatch repair — the same targets need three to four times as many cycles. Two engineering conclusions follow. **Mismatch repair is the first thing to disable**, which is why MAGE strains are mismatch-repair deficient and why that in turn raises the background mutation rate, so you sequence the final strain rather than trusting it. And **the oligo design matters more than the cycle count**, since p enters the exponent: targeting the lagging strand, avoiding secondary structure, and using mismatches that evade repair all raise p, and raising p from 5% to 20% saves more cycles than any amount of patience.

The connection to part 2 chapter 15 is the reason MAGE exists at all. Control is distributed, so a useful flux improvement needs several changes at once, and the combinatorial space of "which ribosome binding sites, at which of twenty genes" is far too large to build one construct at a time. MAGE generates that diversity in a population and lets a screen or selection find the combination — which is the parallelism argument of part 2 chapter 11 implemented at the chromosome.

Cas9, and the thing to get right about bacteria

SpCas9 cuts double-stranded DNA 3 bp upstream of a 5′-NGG-3′ protospacer-adjacent motif, guided by a 20-nucleotide spacer, leaving blunt ends. Cas12a recognises a T-rich PAM, 5′-TTTV-3′, cuts further from it and leaves staggered ends, and processes its own guide array from a single transcript — which makes it the better choice when you want many guides from one promoter, or when your target region is AT-rich and lacks NGG sites. SpCas9 remains the default for a single well-placed cut because its targeting range is denser in most genomes and its variants are better characterised.

Now the part that is genuinely different from mammalian work. **Bacteria have no efficient non-homologous end joining**, so a chromosomal double-strand break is not repaired and is lethal. There is no "cut and let the cell make an indel": there is cut and die, unless the cell happens to carry a repair template that restores a sequence the guide can no longer recognise. Which makes Cas9's role in bacteria a **counterselection** — it kills everything that has not been edited by recombineering — rather than an editing enzyme. CRMAGE and similar hybrids are exactly this: Red recombineering supplies the edit, Cas9 removes the unedited majority.

Size the benefit. If recombineering edits a fraction f of the population and the Cas9 cut kills a fraction of the unedited cells, the edited fraction among survivors is f/(f + (1 − f)(1 − kill)):

Recombineering frequency f	Cas9 kills 99.9%	Cas9 kills 99.99%
10^{-4}	9% edited among survivors	50%
10^{-3}	50%	91%
10^{-2}	91%	99%

So a counterselection converts an unscreenable 0.1% into a 50 to 91% hit rate, which is the difference between sequencing thousands of colonies and sequencing four — part 1 chapter 10's selection-versus-screening argument, with numbers. And it explains the standard failure: if the "edited" colonies are mostly unedited, the guide is not cutting efficiently, or the cells have escaped by mutating the target, the PAM or the Cas9 plasmid — which is why escapers should be sequenced rather than discarded, since what they tell you is which part of the system failed.

Designing guides, and what to record

For a knockout, place guides in the first third of the coding sequence, so that any survivor carrying a small repair-template-derived change is still disrupted; design three, because guide activity varies several-fold for reasons that are still not fully predictable; and check each against the genome for near-matches, counting mismatches and their positions, since mismatches in the PAM-proximal seed region abolish cutting while distal ones may not. Record every PAM site in the target region and the predicted off-targets for each guide, because that record is what a reviewer or a supervisor will ask for.

Then the repair template, which in a bacterium is not optional: a double-stranded cassette or a single-stranded oligo carrying the edit, with homology arms, and — this is the detail that is frequently missed — **designed so that the edited sequence can no longer be cut**. Either the edit itself destroys the protospacer or the PAM, or you must introduce a silent change that does. Otherwise Cas9 cuts the repaired allele and the edited cells die along with the rest.

Lambda Red	`Exo chews 5′, Beta anneals, Gam blocks RecBCD; 40–50 bp arms suffice`
MAGE accumulation	`P(all k loci after N cycles) = [1 − (1 − p)^N]^k`

MAGE numbers	`p = 0.2: 4 loci need 10 cycles for 64%, 10 loci need 20 for 89%`
PAMs	`SpCas9 NGG, blunt cut 3 bp upstream; Cas12a TTTV, staggered, self-processing array`
Cut ≠ edit	`no NHEJ in bacteria, so an unrepaired cut is lethal: Cas9 is the counterselection`
Enrichment	`edited among survivors = f/(f + (1−f)(1−kill))`

Three things people get wrong

- ‘CRISPR edits the genome.’ In bacteria it cuts, and the cut kills; recombineering does the editing and Cas9 selects for it.
- ‘More MAGE cycles will get there.’ p is in the exponent — improving the oligo from 5% to 20% per cycle saves more than tripling the cycles.
- ‘The repair template restores the gene, so we are done.’ If the repaired allele still matches the guide and PAM, it gets cut too. Destroy the site with the edit.

Self-check — answer on paper first

1. How many MAGE cycles are needed for 80% of the population to carry edits at six loci, at 20% per cycle?

Require $[1 - 0.8^N]^6 \geq 0.8$, so $1 - 0.8^N \geq 0.8^{1/6} = 0.9637$, giving $0.8^N \leq 0.0363$ and $N \geq \ln(0.0363)/\ln(0.8) \approx 14.8$, so fifteen cycles. The useful corollary is the sensitivity: at 10% per cycle the same target needs about 32 cycles, so the oligo design decides whether this is two weeks of work or five.

2. Explain why Cas9 is lethal in *E. coli* but produces indels in a mammalian cell line.

Because the repair capability differs. Mammalian cells have efficient non-homologous end joining, which rejoins a broken chromosome imprecisely and produces the insertions and deletions that are the usual editing outcome. Most bacteria lack an efficient NHEJ pathway, so a chromosomal double-strand break cannot be rejoined and the cell dies — which makes the cut a selection rather than an edit, and means a repair template must be supplied for anything to survive.

3. Your CRISPR–recombineering experiment gives plenty of colonies and almost none are edited. Name the three explanations and how to distinguish them.

The guide is not cutting, so there is no counterselection — test by transforming the guide plasmid without a repair template, which should kill nearly everything if cutting works. The cells have escaped by losing or mutating the Cas9 or guide plasmid — test by re-isolating plasmid from survivors and sequencing the guide cassette. Or the recombineering frequency is far below what you assumed, so even good killing leaves few edited cells among survivors — test by applying a conventional selectable marker edit in parallel to measure f directly. The enrichment table above tells you which is plausible: with good killing, f must be very low indeed to give this result.

What to open a book for. Snyder & Champness on recombination, selection and counterselection, which remains the best treatment of why selection design matters. Then Datsenko & Wanner, PNAS 97:6640 (2000) for the one-step workflow and Wang et al., Nature 460:894 (2009) for MAGE, plus a 2023-or-later CRISPR-in-metabolic-engineering review.

Genome engineering II: CRISPRi, base editing, biosensors and dynamic control

Regulating a gene is not editing it, and for an essential gene it is the only option you have. This chapter is about the tools that change expression without changing sequence — and about closing a loop with them.

CRISPR interference, and why it beats a knockout

Mutate the two nuclease domains of Cas9 and you get dCas9: a programmable DNA-binding protein that still finds its target and no longer cuts. Bound at a promoter or in the early coding sequence, it blocks RNA polymerase — initiation if it sits on the promoter, elongation if it sits in the gene — and the gene is repressed. The repression is **tunable**, through the guide's affinity and position and through dCas9 expression level; **reversible**, by removing the inducer or curing the plasmid; **multiplexable**, since many guides can share one dCas9; and it requires no change to the chromosome, which is why it works in hosts whose editing tools are poor — the Cleto study in *C. glutamicum* being the standard demonstration.

Three reasons it beats a knockout, and the first is decisive. **Essential genes cannot be knocked out**, so for the large fraction of central-metabolism genes that are essential — and those are precisely the ones whose flux you want to redirect — a knockdown is the only experiment available. **A dose-response is more informative than an on-off answer**: a graded series of guides tells you the shape of the flux response, which is part 2 chapter 15's control coefficient measured directly. And **it is fast**: a new target is a new 20-base spacer, ordered as an oligo pair, rather than a cassette, a selection and a curing step.

What it cannot do: reach zero. Repression of 90 to 99% is typical and complete silencing is not, so a residual flux remains — which is part 1 chapter 8's leak argument from the other direction, and which matters when the residual activity of an enzyme is enough to spoil a selection. It also acts on transcription, so the protein already present is cleared only by dilution, as part 1 chapter 1 insisted: a knockdown takes several generations to show its phenotype, and in a slow-growing or stationary culture it may never fully show it. And guides can repress neighbouring genes in an operon, so a polar effect must be excluded before a phenotype is attributed.

CRISPR activation is the same machinery with an activator domain fused to dCas9, recruiting polymerase instead of blocking it. It works well in yeast, where eukaryotic activation domains are potent and the available upstream space is large; in bacteria it is more constrained, because activation depends on positioning the fused domain within a narrow window upstream of the promoter and the achievable fold-activation is modest. Treat it as a real tool in yeast and a developing one in bacteria.

Base editing: a change without a break

Fuse a deaminase to a catalytically impaired Cas9 — a nickase, or dead Cas9 — and it chemically converts a base within a small window of the protospacer without ever cutting both strands: cytosine deaminases give C→T, adenine deaminases give A→G, with the edit typically falling in a window of a few bases positioned at a fixed distance from the PAM.

When it beats a double-strand break, which is the question the week asks. When the organism cannot survive the break, which in bacteria is always — a base editor needs no repair template and kills nothing, so the editing efficiency is not multiplied by a survival probability. When you want a specific point change: a feedback-resistant allele of an allosteric enzyme (part 1 chapter 3), a promoter mutation, a premature stop codon to inactivate a gene without deleting it. And when you want to make many such changes at once, since base editors multiplex well and each edit is independent of a template.

What it cannot do: insert or delete anything; reach a target with no suitably positioned PAM, since the editing window's position relative to the PAM is fixed; or avoid bystander edits, because any editable base in the window will be edited — so a target with two cytosines in the window gives a mixture of products. Checking the window for bystanders is a design step, exactly like checking for internal restriction sites in chapter 21.

Biosensors: turning a metabolite into a signal

A transcription-factor biosensor is a natural regulatory protein repurposed. The transcription factor binds your metabolite, which changes its affinity for its operator, which changes transcription of a promoter you have placed in front of a fluorescent protein. The output is fluorescence per cell that varies with intracellular metabolite concentration — and, with that, the test stage of part 2 chapter 11 jumps from hundreds of samples a week on HPLC to 10^7 an hour by sorting.

Metabolite classes with well-characterised sensors include aromatic amino acids and their intermediates, several organic acids including muconate and malonyl-CoA, and sugars and sugar phosphates; plus riboswitch-based sensors for a handful of metabolites that bind RNA directly. The parameters to characterise before trusting one, which are the biosensor equivalent of a calibration curve: the **dynamic range** between uninduced and fully induced, the **operating range** of concentrations over which the output actually varies, the **leak**, and the **single-cell distribution** at each concentration — because, as chapter 22 said, a sensor whose distributions overlap cannot sort however well its means separate.

The failure mode to design against is the one part 2 chapter 11 named: cells optimise what is selected. A sorting campaign against a biosensor returns variants that light up the sensor, including ones that have mutated the sensor, lost the degradation of the fluorescent protein, or found an unrelated metabolite that cross-activates the transcription factor. Confirm every hit by the real assay and report the confirmation rate.

Dynamic control, with the loop drawn

Part 2 chapter 19 argued that a toxic intermediate calls for dynamic rather than static control, because static expression forces one compromise level for the whole run. Now the components exist, and the loop is drawable: a biosensor for the intermediate drives a promoter; that promoter expresses either a small RNA or a dCas9 guide against the upstream enzyme's transcript, or a protease-recognition tag system against the enzyme itself; so as the intermediate rises, the upstream enzyme is throttled, and as it falls, the throttle releases. The downstream half of the loop is the mirror image: the same sensor drives expression of the consuming enzyme, so supply and demand are matched at a set point rather than at a guess.

What you would measure to prove the loop works — and this is the question that separates a control system from a decorated construct. **The intermediate pool itself**, which should hold near a set point in the closed-loop strain and rise or oscillate in the open-loop one; without that measurement you have not demonstrated control, only an outcome. **The actuator's output**, meaning the upstream enzyme's level over time, which should visibly track the sensor. **The response to a disturbance**: pulse the substrate or step the inducer and show that the loop returns the intermediate to its set point, with a settling time you can quote — this is the test any control engineer would demand and almost no paper reports. And **the comparison that matters**: titer and yield against the best static expression level, found by titration, because a loop that beats a badly chosen constant is not evidence of anything.

The stability question is also real and is worth raising before you build: a feedback loop with high gain and a long delay oscillates, and the delays here are substantial — transcription and translation of the actuator, then dilution of the existing enzyme, which part 1 chapter 1 put at a generation time. A loop whose actuator can only act by dilution is a loop with a very slow response, which is one reason degradation tags appear in these designs.

The toolkit, consolidated

Tool	What it does	Choose it when
Restriction–ligation	Joins parts with a scar at a shared site	One subclone into an existing backbone, junction sequence irrelevant
Gibson	Scarless join of a few fragments by overlap	Two to four fragments, any sequence, no rebuild planned
Golden Gate / MoClo	One-pot ordered assembly of many parts with defined 4 bp junctions	Many parts, combinatorial variants, repeated swaps, foundry work
Yeast in vivo assembly	Joins fragments by homologous overlap inside the cell	Many or large fragments; joining transcription units via connectors
Lambda Red recombineering	Replaces a chromosomal sequence using short homology arms	A defined deletion or insertion with a selectable marker
MAGE	Cycled oligos edit many loci across a population without selection	Combinatorial tuning of many loci where a screen or selection exists
Cas9 / Cas12a	Programmable double-strand break — in bacteria, a counterselection	Enriching rare recombineered edits; curing plasmids; killing unedited cells
CRISPRi (dCas9)	Tunable, reversible, multiplexable knockdown	Essential genes; dose–response; hosts with poor editing tools
Base editing	Single-base change with no double-strand break	Point mutations such as feedback-resistant alleles; multiplexed point edits; hosts that cannot survive cuts
Biosensor + sorting	Converts an intracellular metabolite into a sortable signal	Libraries too large to screen by HPLC; and as the sensor in a dynamic loop
Dynamic control	Holds an intermediate at a set point by feedback	Toxic intermediates; growth–production decoupling; mismatched pathway segments

Three things people get wrong

- ‘CRISPRi knocks the gene out.’ It knocks it down, typically by 90 to 99%, and the residual flux is often what decides the experiment.
- ‘Base editing is CRISPR without the mess.’ It cannot insert or delete, needs a PAM at the right distance, and edits every editable base in its window.
- ‘The dynamic circuit raised the titer, so the loop works.’ Without the intermediate measured and a disturbance test, you have an outcome, not a demonstrated controller.

Self-check — answer on paper first

1. Why is CRISPRi the only available experiment for an essential gene, and what is the cost of using it?

Because a deletion of an essential gene cannot be recovered — the cells die — so there is no strain to measure, whereas a knockdown leaves the gene present and reduces its expression to a chosen level, giving a viable strain and a dose–response. The costs are that repression is incomplete, so a residual flux persists and a phenotype may be attenuated; that the effect appears only after the existing protein is diluted out over several generations; and that a guide can exert polar effects on downstream genes in the same operon, which must be excluded before the phenotype is attributed to the intended target.

2. You want a feedback-resistant allele of an allosteric enzyme, changing one codon. Compare base editing with a CRISPR–recombineering edit.

Base editing is likely the better route if the required change is a C→T or A→G transition and a PAM sits at the right distance to put the target base in the editing window: no repair template, no double-strand break, no killing, and it multiplexes if you want the same change in several genes. CRISPR–recombineering handles any substitution, insertion or deletion and does not depend on window positioning, but needs a repair template designed to destroy the protospacer, and its yield is the recombineering frequency enriched by the counterselection. Check the window for bystander editable bases first — that check decides it as often as anything else.

3. Design the minimum experiment that would convince you a dynamic control loop is working.

Three measurements against an open-loop control. The intermediate pool over time, which should sit near a set point in the closed loop and drift or accumulate in the open one. The actuator's level over the same time course, showing it tracks the sensor rather than merely being expressed. And a disturbance: a substrate pulse or an induction step, with the intermediate returning to its set point and a settling time reported. Then the comparison that makes it worth having — titer and yield against the best static expression level found by titration, not against an arbitrary constant.

What to open a book for. Cleto et al., ACS Synth. Biol. 5:375 (2016) for CRISPRi in a non-model industrial host, a metabolite-biosensor review for the sensor parameters, and Hartline et al., Metab. Eng. 63:126 (2021) reread with the tools in hand — it reads differently once you know what the actuators are.

Genome design and non-model hosts: edit counts, error budgets and toolkit arithmetic

Two apparently different questions with the same arithmetic underneath: how many edits can you make before verification becomes the bottleneck, and how many cycles does a new host's toolkit have to repay?

Why rewrite a genome

Fredens and colleagues synthesised a 4 Mb *E. coli* genome in which two serine codons, TCG and TCA, were replaced by AGC and AGT, and the TAG stop codon by TAA — **18,214 edits** in total, compressing the genetic code from 64 to 61 codons. The purposes are three and they are worth separating.

- **Free codons for new chemistry.** A codon with no assigned tRNA or release factor can be reassigned to a non-canonical amino acid, which is how you put chemistry into a protein that the twenty natural residues cannot do.
- **A barrier against viruses and foreign DNA.** A phage genome arriving in a recoded host carries codons the host can no longer translate — which is part 4 chapter 52's restriction-barrier logic implemented in the code itself rather than in a nuclease.
- **A test of what is essential.** Building a genome from a design is the only way to find out whether the design was right, which is a different kind of knowledge from deleting genes one at a time.

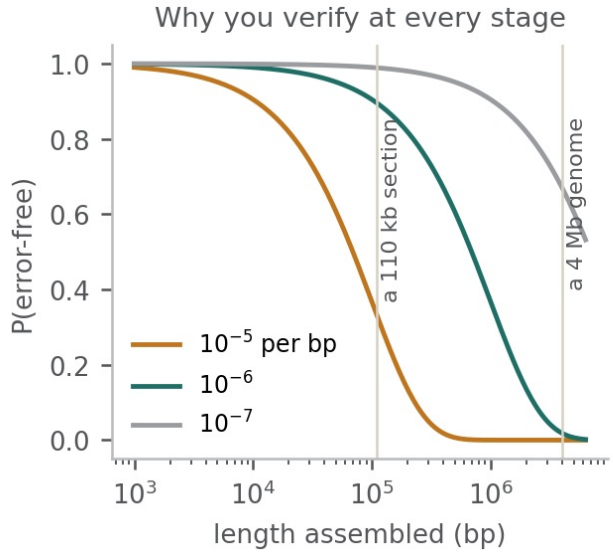
The costs are visible in the same paper and should be quoted alongside: 18,214 edits is an enormous design and verification burden; overlapping genes, where one gene's coding sequence runs into another's on the other strand, had to be corrected by hand because a synonymous change in one frame is not synonymous in the other; and the resulting strain grew more slowly than its parent, which is the honest summary of what recoding costs a cell.

Counting the edits, and the error budget

Do the arithmetic the week asks for. 18,214 edits across about 4 Mb is **4.6 edits per kilobase** — roughly one per 220 bases, which means almost every synthesised fragment carries several. Against roughly 1.3 million codons in the genome, the edits touch about **1.4% of all codons**. And built in sections of 91 to 136 kb, a 4 Mb genome is about **37 sections**, each of which must be assembled, verified and installed.

Now the part that explains the whole build strategy. If synthesis and assembly leave a residual error rate r per base pair, the probability that a sequence of length L is error-free is approximately e^{-rL}:

Residual error rate	P(a 110 kb section is clean)	P(a 4 Mb genome is clean)
10^{-5} per bp	33%	4×10^{-18}
10^{-6} per bp	90%	1.8%
10^{-7} per bp	99%	67%



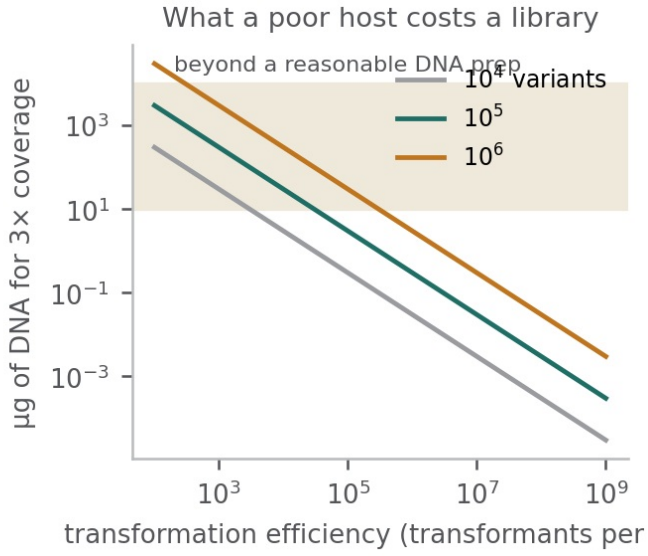


Figure 25.1 Left: probability that an assembly is error-free against its length, for three residual error rates, with a 110 kb section and a 4 Mb genome marked. Right: micrograms of DNA needed for 3× coverage of a library against the host's transformation efficiency, with the region beyond a reasonable preparation shaded. Computed for this module.

The conclusion is forced rather than chosen: **at any achievable error rate, a whole genome assembled without intermediate verification is certain to carry errors, while a 110 kb section is usually clean.** So the build hierarchy — oligonucleotides to fragments to sections of about 100 kb, with sequence verification at every stage, then replacement of the natural genome piece by piece with selection and counterselection — is not caution, it is the only design that can work. The same logic scales down to your own projects: verify at the level where the probability of being clean is high, not at the end where it is not.

Minimal genomes get one paragraph and it is the important one. **"Essential" depends on the conditions.** A gene is dispensable in rich medium at 37 °C and essential in minimal medium, or under osmotic stress, or at 42 °C, or in a strain carrying your pathway. So a minimal genome is minimal for something, and the strains produced this way are typically fragile outside the conditions they were minimised in — which is a feature if the condition is your process and a liability if it is not.

Choosing a non-model host, with the two calculations

The reasons to leave *E. coli* are the host properties of part 6: growth rate, tolerance, substrate range, native pathways, secretion and regulatory status. The reason not to is that everything has to be rebuilt. Two calculations decide it.

The toolkit repayment. Suppose a faster host shortens your design–build–test cycle from five days to four — a 20% gain, which for *V. natriegens* doubling in roughly half *E. coli*'s time is plausible. Building the toolkit first — transformation method, replicon, markers, characterised promoters, an editing tool, curing, a reporter — costs perhaps six months. At one day saved per cycle, that is **180 cycles to break even**. Unless your project will run many hundreds of cycles, or unless someone else has already built the toolkit, the arithmetic says stay where the tools are. This is the calculation that most enthusiasm for new hosts does not survive, and it is worth doing explicitly because the conclusion flips entirely when a toolkit already exists: then the cost is weeks, not months, and the same 20% gain is worth taking.

The library coverage. Transformation efficiency decides whether a library approach is possible at all, as part 1 chapter 10 said. For 3× coverage of a 10^5-member library you need 3×10^5 transformants: at 10^8 per microgram that is 3 ng of DNA, at 10^6 per microgram it is 0.3 µg, and at 10^4 per microgram it is 30 µg — a preparation that is possible but painful, and at 10^3 it becomes 300 µg, which is not a library experiment. So a host's transformation efficiency does not merely slow you down; below a threshold it removes an entire class of experiment, and that is the number to measure in week one rather than week ten.

Host	Attractive because	What it lacks
E. coli	Fastest cycle, every tool, deepest literature	No secretion to the medium, endotoxin, no GRAS status, acetate overflow
Vibrio natriegens	Divides about twice as fast as *E. coli*, naturally competent, grows on formate and acetate	A young toolkit, salt requirement, less literature, unfamiliar physiology
C. glutamicum	Native amino-acid export, high tolerance, GRAS, little overflow	Slower growth, fewer parts, conjugation or electroporation with modest efficiency
Y. lipolytica	Oleaginous, Crabtree-negative, hydrophobic substrates, secretory	NHEJ-dominated targeting, dimorphism, fewer characterised parts
C. necator	PHA physiology, autotrophic growth on H_2 and CO_2	Slow growth, conjugation-based delivery, restriction barriers in wild isolates
Streptomyces	Native secondary metabolism, secretion, large biosynthetic gene clusters	Mycelial growth, slow cycles, conjugation only, GC-rich sequence design

And the answer to the week's last framing question — when to stay in *E. coli* and move only the final strain: when the project is an iteration-heavy design problem rather than a physiology problem. Build, tune and screen in the host with the shortest cycle, then transfer the finished pathway into the production host once and accept that it will need re-tuning there. The cases where that fails are the ones where the host's physiology is the point: you cannot prototype a PHA accumulation strategy in a host that does not accumulate, or a secretion strategy in a host that does not secrete.

A toolkit checklist, and three first experiments

For any host new to you, the checklist is: a transformation method with a measured efficiency; a replicon that is maintained, or an integration site; a selectable marker that works in that organism's antibiotic-resistance background; a set of characterised promoters and ribosome binding sites with measured strengths in your conditions; an editing tool; a way to cure the plasmids you used; a reporter that can be read in that host's background fluorescence; and an

annotated genome, ideally including its restriction–modification systems from part 4 chapter 52.

The first three experiments, each with a control and a pass mark, because a plan without pass marks is a wish. **One**: transformation efficiency with a known replicating plasmid, controlled against a no-DNA mock, with a pass mark of 10^5 transformants per microgram — above that libraries are possible, below it they are not. **Two**: expression of a fluorescent reporter from two or three promoters, controlled against an empty vector for background, with a pass mark of a clearly resolved signal and a measured fold-range between the promoters. **Three**: a single targeted edit — a deletion of a non-essential, easily scored gene — controlled against an untargeted template, with a pass mark of verified edits among a handful of screened colonies.

And what would make you abandon the host: transformation efficiency stuck below about 10^3 per microgram after the obvious fixes, which eliminates library work and makes every construct a project; no working marker or no plasmid maintained; a doubling time so long that the cycle exceeds the project's patience; or restriction barriers that defeat every bypass in part 4 chapter 52. Writing those criteria down in advance is what lets you abandon a host after three months instead of eighteen.

Recoding scale	`18,214 edits in 4 Mb = 4.6 per kb ≈ 1.4% of codons; ~37 sections of 91–136 kb`
Error budget	`P(clean) ≈ e^(−rL): 90% for 110 kb at r = 10⁻⁶, 1.8% for 4 Mb`
Build rule	`verify at the length where P(clean) is high, not at the end`
Toolkit repayment	`(months to build × days) ÷ (days saved per cycle) = cycles to break even; ~180 for a 20% gain`
Library feasibility	`3N/efficiency = µg of DNA; 10⁵ variants needs 0.3 µg at 10⁶/µg, 30 µg at 10⁴/µg`
"Essential"	`conditional on medium, temperature and what the strain is carrying`

Three things people get wrong

- ‘We'll sequence the final construct.’ At 10^{-6} per base a 4 Mb assembly is clean 1.8% of the time. Verify in 100 kb pieces.
- ‘A faster host speeds up the project.’ Only after the toolkit repays itself, which at a 20% cycle gain takes of order 180 cycles.
- ‘A minimal genome is a clean chassis.’ It is minimal for the conditions it was minimised in, and frequently fragile outside them.

Self-check — answer on paper first

1. At a residual error rate of 10^{-6} per base, what fraction of 50 kb assemblies are error-free, and how does that justify the build hierarchy?

$e^{-0.05}$ = 95%, so nineteen of twenty are clean and a single verified clone is usually obtainable from a handful. At 500 kb it falls to 61% and at 4 Mb to 1.8%, so the probability of success collapses with length while the cost of a failed assembly rises. Building in pieces of tens to low hundreds of kilobases and verifying each one means every stage has a high success probability and every error is caught where it is cheap to fix — which is the same logic as verifying a Golden Gate transcription unit before assembling it into a pathway.

2. Your group proposes moving to a host with a 15% faster cycle and no existing toolkit. Make the argument for or against with numbers.

Against, unless the project is unusually long. A 15% gain on a five-day cycle saves 0.75 days per cycle; a toolkit of transformation, markers, characterised parts, editing and curing is realistically four to six months, say 150 days of work, so break-even is around 200 cycles — far more than most PhD-scale projects run. The arguments that would flip it: a property other than speed that the new host has and *E. coli* cannot be given, such as native secretion or a substrate range; an existing toolkit from another group, which cuts the investment from months to weeks; or a plan to use the host for many projects, which amortises the toolkit across all of them.

3. Why does transformation efficiency remove a class of experiment rather than merely slow it down?

Because library-based approaches require covering the library, and coverage is transformants divided by library size: at 3× coverage of 10^5 variants you need 3×10^5 transformants, which at 10^4 per microgram means 30 µg of DNA per experiment and at 10^3 means 300 µg — quantities that are not preparable for routine work. Below that threshold, every variant must be built and tested individually, so the whole parallelism argument of part 2 chapter 11 disappears and the project becomes rational design only, which also means it needs a far better prior.

What to open a book for. Fredens et al., Nature (2019), read for the design rules and the edit count rather than the

methods. The *V. natriegens* preprint and the 2024 PNAS Nexus paper on natural plasmid transformation for the fast-host case, and the SEVA broad-host-range vector literature for what a transferable toolkit looks like. Part 4 chapter 52 of this module is the chapter to read beside this one.

Closing notes to part 7

The six weeks in one page

Week 20. Methods differ in what they leave at the junction and whether the construct can be taken apart again. Gibson is scarless and good to four or five fragments; Golden Gate works because a correct join destroys the recognition site; MoClo adds the syntax that makes parts interchangeable and robots possible. Restriction–ligation survives for the single subclone where no junction matters.

Week 21. A 3 kb part has a 77% chance of containing a six-base site and needs domesticating. Of 256 four-base overhangs, 16 are palindromic, 120 are mutually non-complementary, and imposing a two-base minimum distance against every other overhang and its complement leaves **24** — which is why published high-fidelity sets are that size. Eight transcription units at 80% implies 97.2% per junction; the independence model gives the floor, not the answer. Assemble in silico and translate every frame before ordering.

Week 22. Efficiency comes from the standard-curve slope, and assuming it is perfect overstates a threefold ΔC_t change by 17% at E = 1.90 and 50% at E = 1.75. A band is evidence of size and antibody binding, never of activity. A mean fluorescence shift and a bimodal split look identical in a population average and completely different in a sort.

Week 23. Exo, Beta, Gam, and 40–50 bp arms. MAGE accumulates as $[1 - (1 - p)^N]^K$, so four loci need ten cycles at 20% and the oligo design beats the cycle count because p is in the exponent. In bacteria an unrepaired cut is lethal, so Cas9 is a counterselection: it turns a 0.1% edit frequency into a 50 to 91% hit rate among survivors.

Week 24. CRISPRi is the only experiment available for an essential gene and reaches 90–99% rather than zero. Base editing needs no break and no template but cannot insert, delete or avoid bystanders in its window. A dynamic loop is demonstrated by the intermediate's set point and a disturbance test, not by a titer.

Week 25. 18,214 edits, 4.6 per kb, 1.4% of codons, 37 sections. At 10^{-6} per base a 110 kb section is clean 90% of the time and a 4 Mb genome 1.8%, which forces verification at every stage. A 20% faster host with no toolkit takes about 180 cycles to repay, and a transformation efficiency below 10^4 per microgram removes library work altogether.

On the numbers in this part

The site-frequency, overhang-set, junction-success, qPCR-efficiency, MAGE-accumulation, Cas9-enrichment, error-budget and library-coverage figures were all computed for this module from the assumptions stated beside them. Four carry most of the weight and should be checked against your own system: the per-cycle MAGE replacement frequency, which varies several-fold with oligo design and strain; the Cas9 killing efficiency, which varies with guide and promoter; the residual error rate of your synthesis and assembly supplier; and your host's actual transformation efficiency, which is the first thing to measure and the number most often quoted from a catalogue rather than a bench. The overhang-set result of 24 is a greedy construction under a stated distance rule, so a better search may find a slightly larger set; the order of magnitude is the point.

What remains

Written so far: weeks 1 to 25 and 45 to 61 — 42 of 66. What is left is weeks 26 to 44, which is the parts-and-circuits block (26 to 31), the data and foundry block (32 to 34), the modelling-software block (35 to 38), and the design-algorithms, machine-learning and fluxomics block (39 to 44); plus the professional weeks 62 to 66. I will keep going in tracker order and take them in those blocks, which are the natural groupings — parts and circuits next.